COM-480 · DATA VISUALIZATION · EPFL · SPRING 2026

How sleep quality shapes *student grades*.

Process book — from the original sketches to the final site.

Ali Benchekroun 329911
Elio Rafoul 359387
Hana El Moutaoukil 340995

A short report on how a project we expected to be "more sleep = better grades" turned into a project about why hours of sleep barely move the needle at all — and how we redesigned every chart, every word and every colour of the site around that finding.

§1 Starting point

When we picked the *Student Insomnia and Educational Outcomes* dataset (Mendeley DOI 10.17632/5mvrx4v62z.3) back in March, we expected an obvious story. The dataset is 996 student survey responses across 16 categorical variables — demographics, sleep habits, daytime symptoms, lifestyle factors and self-reported academic outcomes. We assumed the visualisations would essentially show *students who sleep less get worse grades*, and our job would be to draw that relationship clearly.

The three research questions we filed in Milestone 1 reflect that assumption:

- **RQ1** Do students with poor sleep quality also report more concentration difficulties, daytime fatigue and lower academic performance?
- **RQ2** Is there a visible relationship between the number of hours slept per night and perceived academic performance?
- **RQ3** How do screen time before bed, caffeine consumption, stress and physical activity correlate with sleep disturbances?

We split the work into three roles for Milestone 2. **Ali** built the website skeleton and wrote the four chart-drawing functions in D3 v7; **Elio** designed the layout grid and the colour system; **Hana** wrote the Milestone 2 report and started the survey-spec reference table we would later expand for data validation. The submitted prototype already drew all four core charts — the sleep-quality × performance heatmap, a sleep-duration distribution bar, a duration × performance grouped bar and the lifestyle parallel coordinates — each backed by *simulated data* hand-tuned to approximate what we believed the real dataset would look like. Three further visualisations (Sankey, radar, linked scatter + brush) sat lower on the page as "planned enhancement" cards. That decision to simulate instead of load is what set us up for the pivot in Milestone 3.

§2 Milestone 2: sketches and what we got wrong

We submitted five chart sketches with the Milestone 2 report (Figures 1–5 of that document, rendered in D3 on top of the simulated data so they could double as design mock-ups): the sleep-quality × performance heatmap, the duration × performance grouped bar, the lifestyle parallel coordinates, the Sankey flow as a stretch goal, and the radar comparison as a second stretch goal. The grader feedback was encouraging, but two issues surfaced once we plugged in the real CSV at the start of Milestone 3.

Two mistakes baked into the prototype

1. Label drift. Our heatmap legend used *Poor / Fair / Average / Good / Excellent* for sleep quality; the real survey labels are *Very poor / Poor / Average / Good / Very good*. The duration buckets were equally off: the prototype split sleep into `<5h` / `5–6h` / `6–7h` / `7–8h` / `>8h`, while the real survey uses `Less`

`than 4 hours / 4--5 hours / 6--7 hours / 7--8 hours / More than 8 hours` — a different cutoff at the short end and no `5--6 hours` band at all.

2. Simulated data hid the real story. The placeholder arrays we wrote for the grouped bar, the heatmap and even the parallel-coordinates lines all encoded the same prior — short sleep produces bad grades, good sleep produces good grades, and stress and screen time track sleep quality. When we loaded the real CSV we found the opposite for the duration axis (proportions barely shift between bars) and a much messier off-diagonal pattern in the heatmap.

We took those mistakes seriously enough to make data validation the very first task of Milestone 3. Hana wrote a one-page reference sheet listing every column, every category and the spec count from the source dictionary, and Ali wrote a parser in `website/js/data.js` that loads the CSV and *whitelists* every category. Any row whose value does not match the spec is dropped with a console warning. With our reference sheet in hand the whitelist passes 996/996 rows on first run.

We took that strict parser as the contract for the rest of the project. Every chart receives normalised `{raw, ord}` pairs for ordinal fields, so sorting and scaling use the integer ord while tooltips and labels show the original survey wording. We promised ourselves we would not invent values again. The original Milestone 2 prototype is preserved verbatim at `website-m2/index.html` in the repository, so the evolution between the two versions is fully traceable side by side; the rebuilt Milestone 3 site occupies the `website/` folder.

§3 The pivot: what 996 students actually said

Plugging in the real data changed the project. The headline numbers were so different from what we had assumed that, after a long Friday session re-running the cross-tabulations, we stopped to rewrite the project framing on the whiteboard above Ali's desk.

The headline numbers (n = 996)

90%	sleep **7 hours or more**
46%	rate their sleep **very poor** or **poor**
93%	report **high** or **extremely high** stress
88%	rate their grades **poor** or **below average**
4%	rate their grades **good** or **excellent**

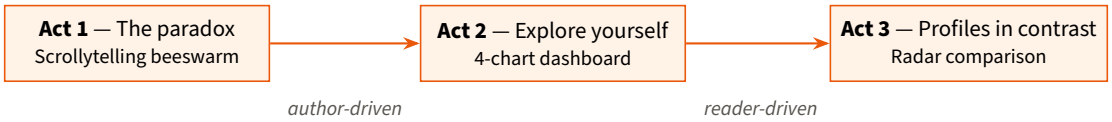
The diagonal we expected in the heatmap of sleep quality vs academic performance turned out to be a kink with most of the mass crammed in the bottom-left corner. Worse, when we cross-tabulated sleep *duration* with performance, the proportions in the 7–8 h and $>$8 h bars looked indistinguishable from the $<$4 h bar. Sleeping more was not associated with better grades. Hana ran a quick χ^2 test in pandas to make sure we were not misreading the cross-tabulation. The duration–performance relationship came back as non-significant; the quality–performance relationship strongly significant.

The project was no longer "more sleep equals better grades". It was the opposite: **hours of sleep don't predict grades; quality and lifestyle do.** That paradox became the spine of the final site. We wrote it on the whiteboard as the one sentence every design decision had to respect:

A first-time reader should understand the paradox in under 30 seconds.

§4 Designing the data story

We needed structure. The course's storytelling lecture introduced the *martini-glass* pattern (Segel & Heer): a single author-driven sequence that opens into a wide reader-driven exploration. That fit our paradox exactly — hook the reader with a guided revelation, then hand them the controls.



Act 1 — the paradox. A scrollytelling sequence that re-arranges and re-colours the same 996 dots six times. The reader scrolls down the left rail; the chart on the right reshuffles in place. Each step adds one fact: the cohort sleeps enough, the cohort sleeps poorly, the cohort is stressed, the cohort under-performs, and finally — hours don't explain grades.

Act 2 — explore yourself. The free-exploration dashboard: heatmap with a chi-square residual toggle, 100 %-stacked horizontal bar for duration × performance, Sankey from duration through quality to performance, and a six-axis parallel-coordinates chart with per-axis brushing. All four charts share a single filter store.

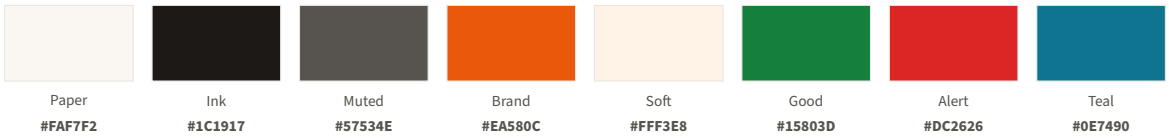
Act 3 — profiles in contrast. A radar with two paired group pickers, defaulted to *very poor sleepers vs very good sleepers*. The user can swap in any other split.

The order matters. Act 1 earns the reader's attention by surprising them with the paradox. Act 2 invites them to verify it themselves. Act 3 lets them dig into specific contrasts. If we had opened with the dashboard, the paradox would have drowned under the controls.

Rewriting the headline. Our first hero draft was punchy — *"They sleep enough. They feel terrible. Their grades are worse."* It sounded more like a tabloid than a survey report, and Hana flagged in review that the italic on *terrible* felt judgmental of students who are already burnt out. We tested four alternatives in a shared doc and settled on the simpler, explicit version: **How sleep quality shapes student grades.** The subtitle names every variable the page uses in one sentence, so a reader can size up the project before scrolling.

§5 Visual identity

The biggest design change between Milestones 2 and 3 was the *look* of the site. The Milestone 2 prototype shipped on a dark GitHub-style background — fast to build, kind to late-night sessions, and a palette Ali had used in a previous project. After we showed an internal demo to two friends outside the course, both used the same word: *admin*. The story is for students and educators, not for engineers; we needed the page to look more like *The Pudding* or *Our World in Data* than like a dashboard.



Elio led the visual rebuild over a long weekend:

- **Warm cream paper** (`#FAF7F2`) instead of charcoal, with a very faint orange-and-violet radial wash on `body::before` for depth.
- **Deep brand orange** (`#EA580C`) as the only primary accent — wordmark, links, italic emphasis in headlines, active toggle states, focus rings and CTA buttons.
- **Fraunces** for every headline, stat number and chart card title, with its variable optical-size axis pinned to `opsz 144` for proper display contrast. The italic cut handles emphasis in the title and takeaways.
- **Inter** for body, UI, axis labels and tooltips. **JetBrains Mono** for very small data labels ("`Step 3 of 6`", radar deltas).

Killing the rainbow. The perception lecture forced us to look hard at the palettes inside the charts. Our M2 parallel coordinates coloured students by sleep quality with a five-step rainbow (`#f78166 / #e3b341 /`

`#8b949e / #58a6ff / #3fb950`). That was wrong: sleep quality is *ordinal*, and rainbow palettes invent category boundaries between adjacent levels. We replaced every ordinal axis with a single-hue sequential palette — Blues for sleep quality and performance, Oranges for stress, Purples for duration, Reds for caffeine, Greys for screen time, Greens for physical activity. The only diverging palette we kept is the chi-square residual view of the heatmap (RdBu), because positive and negative residuals genuinely differ in kind.

Light-theme knock-on. d3's sequential interpolators (`interpolateBlues`, `interpolateOranges`, …) start their t range at near-white, which disappears on cream paper. We wrapped each interpolator in a small `seq()` helper that clamps the t-domain to roughly $[0.22, 0.95]$ — the lightest ordinal step is now still dark enough to read against the page background, while the darkest step keeps its saturation. We discovered the issue when Hana could not tell apart the bottom two segments of the proportional stacked bar in a screenshot from her laptop.

§6 The charts, one at a time

6.1 Scrollytelling beeswarm — Act 1

The hardest chart to get right. Each dot is one of the 996 students in a `d3.forceSimulation` with `forceX`, `forceY` and `forceCollide`. As the user scrolls, an `IntersectionObserver` watches the six step cards in the left rail. When a step enters the middle 20 % of the viewport (`rootMargin: '-40% 0px -40% 0px'`), we update the simulation's `forceX` to cluster the dots by a new variable, then transition the fill on every circle over 700 ms.

Three details we had to revisit during review:

- Our first version used a tight `gap: 75vh` between step cards, which felt like scrolling cross-country. We dropped the gap to `42vh` and reduced the active state to a subtle border + soft orange shadow + a 3 px accent rule on the left edge. The active step is clearly current without dominating the reading.
- We added a **colour-legend strip inside the canvas card**, above the SVG — a gradient bar with the low/high category labels, rebuilt on every step change from the same d3 scale the dots are using. Until we added it, our test reader (Ali's flatmate) could tell that the dots were re-colouring but not by *what*.
- The step-0 cluster looked lost in the white card because our initial `radiusForCount()` was too conservative. Bumping the radius coefficient from `0.16` to `0.21` produces a cluster that fills enough of the canvas to feel like the "996 students" the caption claims.

6.2 Heatmap — RQ1

A 5×5 grid; rows are sleep quality, columns are performance. Cells are coloured by count (Blues sequential) by default, with a toggle to switch to chi-square *standardised residuals* (RdBu diverging). The residual mode is where the story lives: positive residuals at *Average* × *Average* ($+5.1\,\sigma$) and *Average* × *Good* ($+4.6\,\sigma$) reveal a clean diagonal the raw counts hide. The reverse encoding also surfaces an off-diagonal at *Very good* × *Poor* ($+2.9\,\sigma$): a small cohort that sleeps well but still rates their grades low.

Cell-label colour is luminance-aware. The helper picks `#ffffff` on darker fills and `#1C1917` on pale ones, so the digits stay legible as the user toggles between counts and residuals.

6.3 $100\,\%$-stacked bar — RQ2

We replaced the *grouped* bar we had sketched in Milestone 2 with a *100 %-stacked horizontal* bar. Grouped bars sized by absolute count drown the short-duration buckets in the long-duration ones, because 90 % of the cohort sits in 7–8 h or >8 h. Proportional stacking normalises that out — the reader can compare segment widths between bars and see that the performance mix barely shifts between 4 h sleepers and 8 h sleepers. We kept an *Absolute* toggle as a sanity check that the proportions are not driven by tiny n.

The first version drew the performance legend below the bars in the SVG, which collided with the x-axis title "*Share of students at this duration*". We moved the legend into a small HTML toolbar at the top of the card alongside the Proportion/Absolute toggle. Cleaner, and accessible to screen readers as plain text.

6.4 Parallel coordinates — RQ3

Six axes in left-to-right order — *Stress, Screens, Caffeine, Activity, Sleep, Grades* — so lifestyle inputs flow into the intermediate (sleep) and out into the academic outcome. Lines are coloured by sleep quality on the Blues palette. Each axis carries a ↑ better or ↓ worse direction tag.

The original sketch had no brush. We added a `d3-brushY` to every axis after realising the dropdown filter bar alone could not express the queries we wanted to support ("students with very high stress *and* low activity *and* poor quality"). Brushed ranges emit a `{minOrd, maxOrd}` into the global filter store and surface as dismissible chips in the filter bar; dropdown filters and brushes intersect, never overwrite.

> **Implementation trap: infinite brush loop**
>
> The chart rebuilds its own brushes on every `filter:change` event. The first version dropped into an infinite loop, because the programmatic `brush.move()` we call to repaint an existing brush fires the same `end` event we listen on. The fix was a one-line guard — `if (!ev.sourceEvent) return;` — so only user-driven brush events propagate. We left a long comment in the source explaining the trap, because future us will forget.

A separate fix came late: at narrower viewports the rightmost axis's tick labels (*Excel / Good / Avg / Below / Poor*) were rendered to the *left* of the axis, where the dense line bundle made them illegible. We anchored the last axis's labels to the right of its line — like the first axis — so neither end of the chart ever has labels swimming inside the bundle.

6.5 Sankey — Duration → Quality → Performance

A Milestone 2 stretch goal that we promoted to a core chart, because it answers the central thesis in one image: the thick band leaving "More than 8 hours" routes mostly through "Very poor" sleep quality and ends at "Poor" performance. We use `d3-sankey` with one SVG `linearGradient` per link, interpolating between source-node and target-node colours — uniform-colour links broke the visual continuity in our first try.

The Sankey lives in its own *full-width* row in the dashboard. We initially placed it side-by-side with the stacked bar, but at laptop widths the three-column nodes got crushed and the middle-column labels collided with the left-column duration labels. Giving it the full content width fixed it. We also shortened the column headers to single words (DURATION / QUALITY / PERFORMANCE) and rendered every node label in dark ink with a thick cream halo — the medium-blue middle nodes (Good / Very good) had been completely invisible under our earlier white-text-on-dark-blue rule.

6.6 Radar comparison — Act 3

Five axes: *Stress, Screens, Caffeine, Activity, Fatigue*. The user picks two cohorts via paired dropdowns; the two polygons overlay. We normalise every axis to $0 \ldots 1$ (mean ord ÷ max ord), because stress has 4 levels and the others 5 — without the normalisation the stress polygon would look artificially shorter.

Δ annotations at each vertex show the gap between the groups. Positioning them was fiddlier than we expected: our first pass put each Δ at a fixed $+32$ px offset from the axis title, which for the *top* vertex (Stress) meant pushing the Δ *toward* the polygon centre, where it then collided with the data dot and the OUTER tag. We rewrote the placement to be radial — axis title at $r + 24$, direction tag at $+15$ px further out, Δ at $+30$ px further still, with the order flipped on the top vertex so the stack reads outward in every direction.

The group legend was the last thing to land. Two long values like "More than 8 hours (duration)" can stretch Group A's label past the Group B swatch if the positions are hard-coded. We replaced the literal `x=220` with `getComputedTextLength()` and place Group B with a 32 px gap after A's measured width.

§7 Architecture and tools

The dashboard is wired through a single `state.js` module exporting `setFilter`, `setBrush`, `resetFilters`, `getFilteredRows` and a `d3.dispatch('filter:change','brush:change','select:change')`. Every chart file exposes `init(container)`, subscribes to `filter:change`, and never reads state from anywhere else. The

filter bar mirrors dropdown values and renders brush chips from the same store. Two side effects of this architecture: filter state serialises to a URL hash for *share my view*, and the parallel-coords brush chips compose with dropdown filters cleanly.

We chose plain ES modules with no build step. Importing D3 from a CDN, loading the CSV at runtime and serving the website with `python3 -m http.server` keeps the project reviewable in any browser and avoids the framework-of-the-month problem. We considered a build setup with Vite for hot reload, but the savings on dev cycles (about three or four seconds per refresh) did not justify adding a build artifact a grader would have to install before reviewing the code.

```
website/
├── index.html
├── css/{base,layout,components}.css
├── js/
│   ├── main.js  // boot
│   ├── data.js  // CSV parser, label whitelist
│   ├── state.js  // filter store + dispatch
│   ├── filterBar.js  // sticky UI
│   ├── utils.js  // tooltip, palettes, CSV download
│   └── charts/{scrolly, heatmap, stacked, parallel, sankey, radar}.js
└── data/students.csv
```

§8 What we learned

Three lessons we'll take away from the project:

1. **Simulate carefully or not at all.** Our Milestone 2 simulated data nudged us toward the wrong story. Half a day of exploring the real CSV in a notebook before drawing the sketches would have surfaced the paradox earlier, and the M2 sketches would have been more accurate.
2. **The palette lecture matters.** Replacing our prototype's rainbow with single-hue sequential scales made every ordinal chart easier to read, and forced us to articulate what "good" and "bad" mean for each variable — the direction tags on the parallel and radar axes are a direct consequence.
3. **Design for the screencast.** Forcing ourselves to imagine a 2-minute walkthrough made us pick a strong opening (the paradox) and a clear ending (three takeaways). That decision improved the site for every reader, not only the screencast.

If we had more time we would add a *cohort comparison* feature in the dashboard (lock the current filter as Group A, then change it to define Group B and compare counts side by side), and run a small user-test to refine the scrollytelling cadence — at the moment the steps are still slightly long for one-finger trackpad scrolling.

§9 Peer assessment

The project was split as follows. Each task was discussed in our two weekly syncs and reviewed by at least one other team member before being merged.

Ali Benchekroun **329911**	Technical architecture: `data.js` whitelist parser, `state.js` filter store and event bus, `utils.js` helpers. Implementation of the scrollytelling beeswarm (force simulation + IntersectionObserver), the parallel coordinates with per-axis brushing, and the Sankey. Wiring of the global filter and URL-hash sharing. Local-server setup and the GitHub-Pages deployment.
Elio Rafoul **359387**	Design system rework: cream-paper palette, brand orange, Fraunces + Inter typography. Implementation of the heatmap with chi-square residual toggle and luminance-aware cell labels, and the 100 %-stacked bar with abs/prop toggle. Hero copywriting, take-aways layout, scrollytelling step text and gradient legend. Final two rounds of overlap and legibility polish.
Hana El Moutaoukil **340995**	Dataset validation: counts checks against the survey spec, identification of the label and bucket mistakes in the M2 prototype, χ^2 confirmation of the duration/performance independence. Implementation of the radar comparison with paired pickers and radial Δ positioning. Accessibility pass (SVG aria-labels, focus-visible, direction tags). README updates, the process book and figure curation.

We also met for two scheduled review sessions where we read each other's pull requests, ran the site on each other's laptops at different screen sizes, and signed off on four "polish iterations" that fixed scrollytelling spacing, parallel-coords legend placement, Sankey label fills, and radar endpoint stacking.

Figures referenced throughout this document: the five chart sketches we submitted with Milestone 2 (heatmap, grouped bar, parallel coords, Sankey, radar) — see the Milestone 2 report in this same folder. The final visualisations themselves are best read directly on the live site, where every chart is interactive: `com-480-data-visualization.github.io/student-sleep-quality-academic-success`.